

NABH Document Workflows — Agents Documentation

This document describes every agent in `agents/`, grouped by the document-processing pipeline they belong to, plus how they are orchestrated.

Pipeline overview

Three parallel pipelines exist — one per document type. Each follows the same shape: **compliance check** → **validation** → **PDF generation**, with an optional **audit log** step. Only the Complaint pipeline additionally has **intake** and **extraction** steps, because it can accept raw unstructured text input (the other two only ever receive structured JSON from the web form).

Complaint:	<code>intake → extract → compliance → validate → pdf → audit</code>
Admission Advice:	<code>compliance → validate → audit → pdf</code>
Hospital Pre-Registration:	<code>compliance → validate → audit → pdf</code>

1. Complaint pipeline

`intakeAgent.js`

Role: File reader / entry point.

- `readComplaint(path)` delegates to `readTextFile ( services/shared/fileService.js )` to load the raw complaint file (plain text or JSON) as a string.
- No parsing logic — purely an I/O boundary so downstream agents never touch the filesystem directly.

`extractionAgent.js`

Role: Turns raw text into a structured object.

- `extractComplaintFields(rawText) :`
 - If input starts with `{`, it's already JSON — parsed directly.
 - Otherwise treats input as `Key: Value` line-delimited text, splits on `:`, and builds a lookup map.
 - Maps ~23 known keys (`complaintId`, `patientName`, `uhid`, `complaintCategory`, etc.) into a fixed-shape object, defaulting to `" "` for anything missing.

`complianceAgent.js`

Role: Enforces NABH data-completeness rules against `models/complaintSchema.json`.

- `applyNABHCompliance(extracted)` iterates every field in the schema.
- Blank fields in the `optionalFields` set (email, root cause, corrective action, etc.) stay empty; everything else blank is stamped with the sentinel string `"NOT PROVIDED"`.

`validationAgent.js`

Role: Gatekeeper — decides if the complaint is complete enough to proceed.

- `validateComplaint(data)` scans for any field still equal to `"NOT PROVIDED"`.
- Returns `{ isValid, missingFields }`, used by the API to reject incomplete submissions with `422`.

`pdfAgent.js`

Role: Thin wrapper around the PDF renderer.

- `generateComplaintPDF(data)` calls `createComplaintPDF` from `services/complaint/pdfService.js`. No logic of its own — kept separate for pipeline-step symmetry.

`auditAgent.js`

Role: Compliance trail / logging.

- `logComplaint(data)` writes a structured log entry (via `services/shared/logger.js`) recording `complaintId`, `patientName`, `dateOfComplaint`, `closureStatus`.

2. Admission Advice pipeline

`admissionAdviceComplianceAgent.js`

Role: Same job as `complianceAgent.js`, scoped to `models/admissionAdviceSchema.json`.

- `applyAdmissionAdviceCompliance(extracted)` marks any blank field as `"NOT PROVIDED"` — **all 21 fields are required, no optional exceptions.**
- Handles array-typed fields (`safetyPrecautions`) correctly: an empty array counts as blank.

`admissionAdviceValidationAgent.js`

- `validateAdmissionAdvice(data)` — identical shape to `validateComplaint`, returns `{ isValid, missingFields }`.

`admissionAdviceAuditAgent.js`

- `logAdmissionAdvice(data)` logs `patientFullName`, `uhid`, `dateOfAdvice`, `priorityStatus`.

(PDF generation uses `services/admission-advice/pdfService.js` directly — no separate `pdfAgent` wrapper exists for this pipeline.)

3. Hospital Pre-Registration pipeline

`hospitalPreRegistrationComplianceAgent.js`

Role: Same job, scoped to `models/hospitalPreRegistrationSchema.json`.

- `applyHospitalPreRegistrationCompliance(extracted)` — blank fields become "NOT PROVIDED" **except** the optional set: `emailAddress`, `insuranceCompany`, `tpaName`, `policyId`, `preAuthStatus`, `contactPhone2`.

`hospitalPreRegistrationValidationAgent.js`

- `validateHospitalPreRegistration(data)` — identical shape, returns `{ isValid, missingFields }`.

`hospitalPreRegistrationAuditAgent.js`

- `logHospitalPreRegistration(data)` logs `fullName`, `idNumber`, `proposedDate`, `department`.

(PDF generation uses `services/hospital-pre-registration/pdfService.js` directly.)

Orchestration — how the agents are actually run

`agentRunner.js` (CLI, Complaint pipeline only)

Multiplexes single-step invocations:

```
node agentRunner.js <intake|extract|compliance|validate|pdf|audit> <input-file> [output-file]
```

Useful for debugging/testing one agent in isolation. Only wired up for the Complaint pipeline — Admission Advice and Hospital Pre-Registration have no equivalent per-step CLI runner yet.

`workflows/complaintWorkflow.js`

Runs all 6 Complaint agents in sequence for the full CLI flow: reads a file, produces 3 PDF copies (`Complaint_NABH.pdf` , `Complaint_NABH_Rebuilt.pdf` , `Complaint_NABH_LogoAligned.pdf`), and logs the audit entry.

`workflows/admissionAdviceWorkflow.js` /

`workflows/hospitalPreRegistrationWorkflow.js`

Each reads a JSON sample file, runs compliance → validation (logs missing fields to console if any) → PDF generation → audit log.

`agents/apiAgent.js` **(Express server)**

The REST API entry point (`npm run api` , listens on `http://127.0.0.1:3000`). For each of its three `POST` routes it runs the matching pipeline's compliance → validation (returns `422` with `missingFields` on failure) → audit log → PDF generation, then either streams the PDF back or uploads it to configured cloud storage (`?store=true`).

The Complaint route does **not** call `intakeAgent` / `extractionAgent` (the web form already sends structured JSON) and does **not** call `auditAgent` (only the CLI workflow does). The Admission Advice and Hospital Pre-Registration routes **do** call their audit agents, for parity and a live audit trail on every API submission.

Summary table

Pipeline	Compliance agent	Validation agent	Audit agent	PDF renderer
Complaint	<code>complianceAgent.js</code>	<code>validationAgent.js</code>	<code>auditAgent.js</code>	<code>pdfAgent.js</code> → <code>services/complaint/pdfService.js</code>
Admission Advice	<code>admissionAdviceComplianceAgent.js</code>	<code>admissionAdviceValidationAgent.js</code>	<code>admissionAdviceAuditAgent.js</code>	<code>services/admission-advice/pdfService.js</code> (direct)
Hospital Pre-Registration	<code>hospitalPreRegistrationComplianceAgent.js</code>	<code>hospitalPreRegistrationValidationAgent.js</code>	<code>hospitalPreRegistrationAuditAgent.js</code>	<code>services/hospital-pre-registration/pdfService.js</code> (direct)